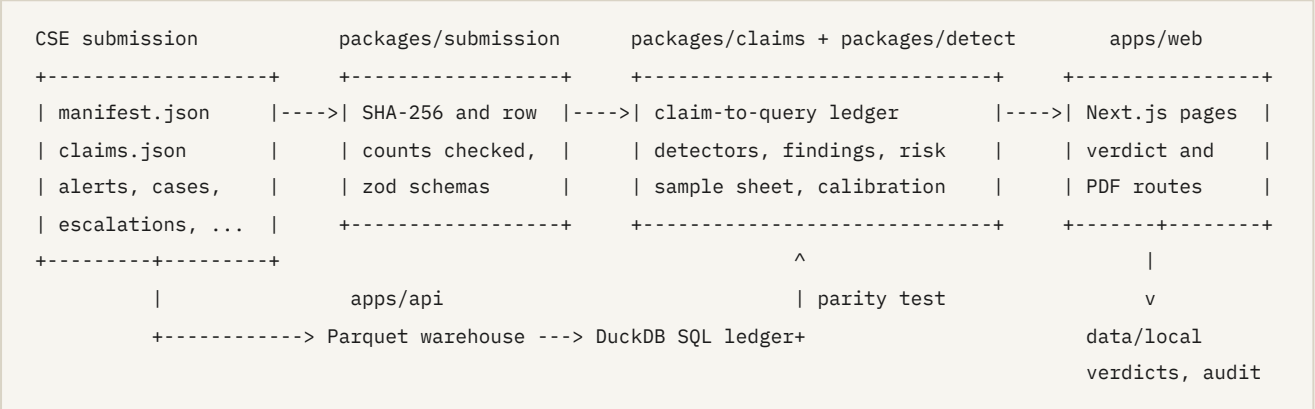
SOC Claim Ledger architecture

For NTRO and NCIIPC. Team 1, Saveetha Engineering College. Target product and working prototype on synthetic SOC data.

What it does

A CSE's quarterly submission to NCIIPC carries two things. The claim side is the self-assessment and KPI report the CISO signs. The evidence side is the SOC's own alert, case, escalation, asset and roster exports. SOC Claim Ledger maps every claim to a query over the evidence, recomputes the figure, and shows the gap with the records behind it one click away. Where a claim reports against a published obligation, such as the NCIIPC Guidelines, the CEA power-sector guidelines or the CERT-In directions, the clause sits beside it. Detectors then look for what a KPI average hides, and the examiner's blind verdicts on a sample of records calibrate every detector.

Data flow



Implemented in the prototype, and used for every number on screen. The simulator in `packages/simulate` runs twelve SOCs for four quarters and writes their submissions and claims files. The TypeScript engine validates each submission, recomputes sixteen questionnaire claims, runs six detectors and writes `data/demo/derived`. The web app renders that output and nothing else, and stores examiner verdicts, the audit log and exported reports under `data/local`. The FastAPI service ingests the same submissions, as newline-delimited JSON or CSV, into Parquet and recomputes the ledger in DuckDB SQL, and a test fails if the two engines disagree.

Proposed for deployment, and not claimed as working at NCIIPC. Ingestion of real CSE exports through a schema-mapping step, the UI reading from the API instead of prebuilt JSON, and signed rule bundles for offline updates.

Components

Part	Role
`packages/submission`	Submission contract. Zod schemas for nine exports, manifest with SHA-256 and row counts, loader that refuses any mismatch
`packages/claims`	Claim schema, the claim-to-query mapping for sixteen metrics, tolerances, gap and insufficient-evidence rules, findings, risk, sampling, calibration
`packages/detect`	SLA bunching, templated notes, negative space against sector peers, weekend coverage, repeat without remediation, versioned registry
`packages/simulate`	SOC simulator with analysts, shifts, queues, SLAs and behaviour toggles, the claims writer, hidden ground truth, simulated examiners
`apps/web`	Next.js App Router UI, self-hosted IBM Plex, verdict, PDF and health routes, pino JSON logs
`apps/api`	FastAPI over DuckDB and Parquet, JSON and CSV ingestion, SQL ledger, parity tests, JSON logs with request IDs, Prometheus metrics
`Dockerfile`, `deploy`	One Docker image built from the root Dockerfile, USB bundle and install script, network-off proof

Logs, health and metrics

Both services log one JSON object per line, the web app through pino on stdout and the API through Python logging on stderr. The API tags each request and every line logged while serving it with an `X-Request-ID`, and its failure bodies carry the same ID. `GET /api/health` on the web app answers 503 until the analysis is on disk, and the Docker health check waits on it. `GET /metrics` on the API serves request counts and latencies by route template, rows ingested and refused ingest runs. A claim the SQL ledger cannot recompute, because an export it needs is missing or its population is empty, is left out rather than reported as zero, the same rule the TypeScript ledger follows.

Offline by construction

No part of the product calls a network. Fonts are bundled through `next/font/local`, telemetry is disabled, and the PDF is rendered by the Chromium already on the machine. The end-to-end test drives every screen and fails on any request that leaves the origin. `make offline-test` runs the same test inside the image with Docker networking removed.

Deployment and hardware

One Docker image holds the web app, the API, the analysis, its data and Chromium. `deploy/offline-bundle.sh` writes the saved image and its checksum to a USB folder, and `deploy/usb-install.sh` verifies and starts it on the examiner's machine. Target machine, to be benchmarked, is 8 CPU cores, 16 GB RAM and 20 GB of free disk, with no GPU and no hosted model. Rules and detectors are versioned, every finding records the version that produced it, and a past assessment can be re-run with the version it used.

Methodology

Each claim holds or fails against its tolerance, 2 points on shares, 10% on mean times, exact on counts. A missing or partial export makes a claim insufficient evidence, which is reported apart from gaps. Detectors are deterministic and versioned, and each finding's weight in the risk indicator is its severity times the precision examiners measured for that detector in earlier cycles. The sample sheet covers every finding before any gets a second record and holds back a counterexample and a seeded random pick. Details are in `docs/methodology.md`.

Validation

Tests hold the simulator deterministic, the clean control free of gaps, a missing export to insufficient evidence, and the SQL and TypeScript ledgers equal for all twelve entities. Calibration today runs on simulated examiners with hidden ground truth, which tests the machinery and not real precision. With NCIIPC, two independent SOC practitioners review blinded samples, and a detector is trusted when their agreement reaches a kappa of 0.7 on an untouched holdout. Details are in `docs/validation.md`.

Offline updates

New rules ship as a versioned bundle on removable media with a SHA-256 manifest. The previous version stays installed, so any past assessment can be reproduced exactly.